

1. Receive Stock from a Purchase Order (Dynamic Lookup)

Your sample data includes an open Purchase Order (**PO-2025-0001**). Let's write a PL/SQL block that dynamically looks up the IDs for the PO line, the receiving dock, and the employee, and then uses your package to receive 25 Wireless Routers into the warehouse.

```
DECLARE
    v_po_line_id NUMBER;
    v_loc_id     NUMBER;
    v_emp_id     NUMBER;
    v_receipt_id NUMBER := NULL; -- Passing NULL tells the package to create a new receipt header
BEGIN
    -- 1. Find the exact Line ID for the Wireless Router on the open PO
    SELECT pl.po_line_id INTO v_po_line_id
    FROM po_line pl
    JOIN purchase_order po ON pl.po_id = po.po_id
    JOIN product p ON pl.product_id = p.product_id
    WHERE po.po_number = 'PO-2025-0001' AND p.sku = 'SKU-1001';

    -- 2. Find the 'Receiving' zone location ID at the East Warehouse
    SELECT location_id INTO v_loc_id FROM warehouse_location WHERE location_code = 'RCV-01';

    -- 3. Find the ID for employee E002 (James Wilson)
    SELECT employee_id INTO v_emp_id FROM employee WHERE employee_code = 'E002';

    -- 4. Execute the actual receipt transaction using your package
    pkg_stock_movement.receive_stock(
        p_po_line_id => v_po_line_id,
        p_location_id => v_loc_id,
        p_qty        => 25,
        p_employee_id => v_emp_id,
        p_receipt_id => v_receipt_id
    );

    DBMS_OUTPUT.PUT_LINE('SUCCESS: Received 25 units from PO-2025-0001!');
    DBMS_OUTPUT.PUT_LINE('Generated Goods Receipt ID: ' || v_receipt_id);

    COMMIT; -- Save the transaction
END;
/
```

```

    v_receipt_id NUMBER := NULL; -- Passing NULL tells the package to create a new receipt header
BEGIN
    -- 1. Find the exact Line ID for the Wireless Router on the open PO
    SELECT pl.po_line_id INTO v_po_line_id
    FROM po_line pl
    JOIN purchase_order po ON pl.po_id = po.po_id
    JOIN product p ON pl.product_id = p.product_id
    WHERE po.po_number = 'PO-2025-0001' AND p.sku = 'SKU-1001';

    -- 2. Find the 'Receiving' zone location ID at the East Warehouse
    SELECT location_id INTO v_loc_id FROM warehouse_location WHERE location_code = 'RCV-01';

    -- 3. Find the ID for employee E002 (James Wilson)
    SELECT employee_id INTO v_emp_id FROM employee WHERE employee_code = 'E002';

    -- 4. Execute the actual receipt transaction using your package
    pkg_stock_movement.receive_stock(
        p_po_line_id => v_po_line_id,
        p_location_id => v_loc_id,
        p_qty         => 25,
        p_employee_id 2   3   4   5   6   7   8   9  10  11  12  13  14  15  16  17
24 25 => v_emp_id,
        p_receipt_id => v_receipt_id
    );

    DBMS_OUTPUT.PUT_LINE('SUCCESS: Received 25 units from PO-2025-0001!');
    DBMS_OUTPUT.PUT_LINE('Generated Goods Receipt ID: ' || v_receipt_id);

    COMMIT; -- Save the transaction
END;
|/ 26 27 28 29 30 31 32 33 34
SUCCESS: Received 25 units from PO-2025-0001!
Generated Goods Receipt ID: 1

PL/SQL procedure successfully completed.

SQL>

```

2. Manual Stock Adjustment (Cycle Count Discrepancy)

Imagine employee Maria Chen (**E001**) does a physical cycle count in Bin **A-01-01** and finds out that 3 Steel Brackets (**SKU-3001**) are damaged and need to be written off. Let's use the **adjust_stock** procedure to safely decrement the stock and log the reason.

```

DECLARE
    v_prod_id NUMBER;
    v_wh_id   NUMBER;
    v_loc_id  NUMBER;
    v_emp_id  NUMBER;
BEGIN
    -- Look up the necessary internal IDs
    SELECT product_id INTO v_prod_id FROM product WHERE sku = 'SKU-3001';
    SELECT warehouse_id INTO v_wh_id FROM warehouse WHERE warehouse_code = 'WH-EAST';
    SELECT location_id INTO v_loc_id FROM warehouse_location WHERE location_code = 'A-01-01';

```

```
SELECT employee_id INTO v_emp_id FROM employee WHERE employee_code = 'E001';
```

```
-- Execute a negative adjustment to write off the broken stock
```

```
pkg_stock_movement.adjust_stock(  
    p_product_id => v_prod_id,  
    p_warehouse_id => v_wh_id,  
    p_location_id => v_loc_id,  
    p_qty_change => -3,  
    p_employee_id => v_emp_id,  
    p_reason_code => 'DAMAGE_WRITE_OFF',  
    p_notes      => 'Items were crushed by a forklift.'  
);
```

```
DBMS_OUTPUT.PUT_LINE('SUCCESS: Stock adjusted successfully for damaged goods.');
```

```
COMMIT;
```

```
END;
```

```
/
```

```
SQL> DECLARE  
v_prod_id NUMBER;  
v_wh_id   NUMBER;  
v_loc_id  NUMBER;  
v_emp_id  NUMBER;  
BEGIN  
-- Look up the necessary internal IDs  
SELECT product_id INTO v_prod_id FROM product WHERE sku = 'SKU-3001';  
SELECT warehouse_id INTO v_wh_id FROM warehouse WHERE warehouse_code = 'WH-EAST';  
SELECT location_id INTO v_loc_id FROM warehouse_location WHERE location_code = 'A-01-01';  
SELECT employee_id INTO v_emp_id FROM employee WHERE employee_code = 'E001';  
  
-- Execute a negative adjustment to write off the broken stock  
pkg_stock_movement.adjust_stock(  
    p_product_id => v_prod_id,  
    p_warehouse_id => v_wh_id,  
    p_location_id => v_loc_id,  
    p_qty_change => -3,  
    p_employee_id => v_emp_id,  
    p_reason_code => 'DAMAGE_WRITE_OFF',  
    p_notes      => 'Items were crushed by a forklift.'  
);  
  
DBMS_OUTPUT.PUT_LINE('SUCCESS: Stock adjusted successfully for damaged goods.');
```

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

```
COMMIT;  
END;  
SUCCESS: Stock adjusted successfully for damaged goods.  
  
PL/SQL procedure successfully completed.  
SQL>
```

3. Verify the Audit Trail

Because your PL/SQL package uses the [apply_inventory_change](#) private procedure, every action you just took was automatically written to the immutable audit log table ([stock_transaction](#)).

Run this standard SQL query to verify that both the Goods Receipt and the Damage Write-Off were logged flawlessly:

```

SELECT transaction_type, qty_change, qty_before, qty_after, reference_type, notes
FROM stock_transaction
ORDER BY transaction_id DESC
FETCH FIRST 5 ROWS ONLY;

```

```

SQL> SELECT transaction_type, qty_change, qty_before, qty_after, reference_type, notes
FROM stock_transaction
ORDER BY transaction_id DESC
[FETCH FIRST 5 ROWS ONLY; 2      3      4

TRANSACTION_TYPE      QTY_CHANGE  QTY_BEFORE  QTY_AFTER  REFERENCE_TYPE
-----
NOTES
-----
ADJUSTMENT              -3           30          27  DAMAGE_WRITE_OFF
Items were crushed by a forklift.

RECEIPT                  25           0           25  GOODS_RECEIPT
Receipt for PO PO-2025-0001

RECEIPT                  120          0          120  INITIAL_LOAD
Opening balance load

SQL> _

```

4. Top Customers by Revenue (Sales Analysis)

Question: Which customers are driving the most revenue based on active or shipped sales orders?

Why it matters: This helps the business identify VIP customers. It joins the **customer** table with the **sales_order** table and aggregates their total spending.

```

SELECT

    c.customer_code,

    c.customer_name,

    COUNT(so.so_id) AS total_orders,

    SUM(so.total_amount) AS total_spent

FROM customer c

JOIN sales_order so ON c.customer_id = so.customer_id

WHERE so.status IN ('CONFIRMED', 'PICKING', 'PARTIAL', 'SHIPPED')

```

GROUP BY c.customer_code, c.customer_name

ORDER BY total_spent DESC;

```
SQL> SELECT
      c.customer_code,
      c.customer_name,
      COUNT(so.so_id) AS total_orders,
      SUM(so.total_amount) AS total_spent
FROM customer c
JOIN sales_order so ON c.customer_id = so.customer_id
WHERE so.status IN ('CONFIRMED', 'PICKING', 'PARTIAL', 'SHIPPED')
GROUP BY c.customer_code, c.customer_name
ORDER BY total_spent DESC; 2    3    4    5    6    7    8    9    10

CUSTOMER_CODE      CUSTOMER_NAME
TOTAL_ORDERS TOTAL_SPENT
-----
CUST-02            NetPro Solutions
      1          2598.8

SQL>
```

5. Active Inter-Warehouse Stock Transfers (Logistics)

Question: What stock is currently requested or in-transit between our different warehouse buildings?

Why it matters: If a warehouse is running low on stock, they can request items from another facility. This query tracks those movements by linking the [stock_transfer](#) header to its lines and resolving the [from](#) and [to](#) warehouse codes.

SELECT

st.transfer_number,

wf.warehouse_code AS from_wh,

wt.warehouse_code AS to_wh,

st.status,

SUM(stl.qty_requested) AS total_items_moving

FROM stock_transfer st

JOIN warehouse wf ON st.from_warehouse_id = wf.warehouse_id

JOIN warehouse wt ON st.to_warehouse_id = wt.warehouse_id

```
SQL> SELECT
      st.transfer_number,
      wf.warehouse_code AS from_wh,
      wt.warehouse_code AS to_wh,
      st.status,
      SUM(stl.qty_requested) AS total_items_moving
FROM stock_transfer st
JOIN warehouse wf ON st.from_warehouse_id = wf.warehouse_id
JOIN warehouse wt ON st.to_warehouse_id = wt.warehouse_id
JOIN stock_transfer_line stl ON st.transfer_id = stl.transfer_id
WHERE st.status IN ('REQUESTED', 'IN_TRANSIT')
GROUP BY
      st.transfer_number,
      wf.warehouse_code,
      wt.warehouse_code,
      st.status;
```

TRANSFER_NUMBER	FROM_WH	TO_WH	STATUS	TOTAL_ITEMS_MOVING
XFR-2025-0001	WH-EAST	WH-WEST	REQUESTED	30

```
SQL>
```